

3. Peterson's Algorithm (2-Process Synchronization)

Syllabus Coverage: Gateway Classes Slides 25–39 | 2-Process Software Solution, Step-by-Step Code Trace, Mathematical Proofs.

3.1 Core Architecture & Shared Variables

- `boolean flag[2] = {false, false};` — Process intention array (`flag[i] = true` means P_i is interested).
- `int turn;` — Decides tie-breaking priority when both processes contend simultaneously.

Peterson's Algorithm: 2-Process Synchronization

State Trace: `flag[0]`, `flag[1]` & `turn` Variable Mechanics

Process P0 ($i = 0$, other = 1)

```
1. flag[0] = true; // Main interested hoon
2. turn = 1; // Pehle aap bhai (P1)
3. while (flag[1] && turn == 1); // Busy Wait
4. /* CRITICAL SECTION */
5. flag[0] = false; // Done, main bahar hoon
6. /* Remainder Section */
```

Key Logic:

P0 expresses interest, gives priority to P1. P0 enters if P1 is not interested OR turn gets updated to 0!

Process P1 ($j = 1$, other = 0)

```
1. flag[1] = true; // Main interested hoon
2. turn = 0; // Pehle aap bhai (P0)
3. while (flag[0] && turn == 0); // Busy Wait
4. /* CRITICAL SECTION */
5. flag[1] = false; // Done, main bahar hoon
6. /* Remainder Section */
```

Key Logic:

P1 expresses interest, gives priority to P0. P1 enters if P0 is not interested OR turn gets updated to 1!

Verification of Peterson's Solution against the 3 Criteria

1. Mutual Exclusion: ✓ YES

Dono ek saath enter nahi ho sakte kyunki turn ek hi integer hai (either 0 or 1, not both!).

2. Progress: ✓ YES

Agar P1 Remainder me hai toh `flag[1]=false` hoga, P0 bina kisi delay ke enter kar jayega.

3. Bounded Waiting: ✓ YES

Waiting bound $k = 1$. Ek process CS se nikalte hi doosre process ko mauka milta hai (No Starvation).

3.2 Step-by-Step Execution Trace

```
// Process P0 Code
flag[0] = true;
turn = 1;
while (flag[1] && turn == 1); // Busy waiting
/* CRITICAL SECTION */
flag[0] = false;
/* Remainder Section */

// Process P1 Code
flag[1] = true;
turn = 0;
while (flag[0] && turn == 0); // Busy waiting
/* CRITICAL SECTION */
flag[1] = false;
/* Remainder Section */
```

3.3 Criteria Proof Summary

Criterion	Status	Mathematical / Logical Proof
Mutual Exclusion	✓ Satisfied	Variable <code>turn</code> cannot be both 0 and 1 at the same nanosecond. Only one process can evaluate while loop to false.
Progress	✓ Satisfied	If P1 is in Remainder, <code>flag[1] == false</code> , allowing P0 to enter immediately without delay.
Bounded Waiting	✓ Satisfied	Bound $k=1$. P0 exiting sets <code>flag[0]=false</code> ; if re-requesting, sets <code>turn=1</code> , granting P1 immediate entry.